

Recommendation Systems for Personalized Content Discovery

Technical Report -- Machine Learning & AI

Harshit Gupta

Enrolment: 24113053

Ishant Bharti

Enrolment: 24115076

Matrix Factorization (SVD) * Collaborative Filtering * ALS * Streamlit Dashboard

■ TABLE OF CONTENTS

Contents

01	Executive Summary Design goals, approach, and key outcomes of the project.
02	Problem Understanding Scale and sparsity challenges of the Netflix Prize dataset.
03	Exploratory Data Analysis Positivity bias, long-tail distribution, and EDA findings.
04	Methodology & Data Pipeline Parquet conversion, temporal splitting, and sparse matrix storage.
05	Model Design ItemCF, ALS, and SVD algorithm architectures.
06	Evaluation Metrics RMSE for rating accuracy; MAP@10 for ranking quality.
07	Experimental Results Benchmark comparison table across all three models.
08	Recommendation Examples Streamlit dashboard demo and SVD explainability.
09	Key Insights & Limitations Memory scalability and cold-start constraints.
10	Future Improvements Hybrid content + collaborative filtering roadmap.

1. Executive Summary

This report presents the design and implementation of a personalized recommendation system using the Netflix Prize dataset -- one of the most celebrated benchmarks in collaborative filtering research. The primary objective was to accurately predict user star ratings and surface a ranked Top-10 list of movie recommendations. Leveraging **Matrix Factorization via Singular Value Decomposition (SVD)** and two complementary collaborative filtering algorithms, we constructed a robust, memory-efficient pipeline capable of processing over 100 million ratings.

2. Problem Understanding

Streaming platforms depend critically on recommendation engines to sustain user engagement. The Netflix Prize dataset poses two core challenges:

- * **Scale:** Over 100 million ratings spanning ~480,000 users and ~17,770 movies.
- * **Sparsity:** Most users have rated only a tiny fraction of the catalog, creating a highly sparse user-movie interaction matrix that strains conventional similarity methods.

A successful model must discover latent patterns between users and movies without exceeding available memory, while still generalising to unseen user-item pairs.

3. Exploratory Data Analysis

Prior to modelling, we performed thorough exploratory data analysis (EDA) to characterise user behaviour and dataset properties.

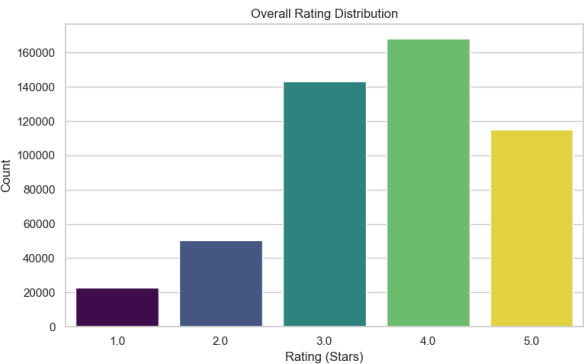


Figure 1: Overall Rating Distribution

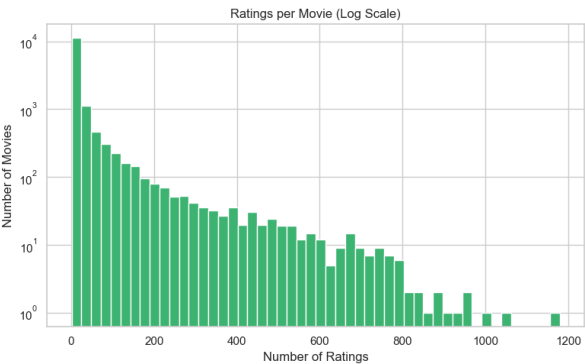


Figure 2: Ratings per Movie (Log Scale)

As shown in Figure 1, users exhibit a strong **positivity bias** -- ratings of 3, 4, and 5 stars dominate the distribution, requiring models to avoid the degenerate solution of always predicting 4 stars. Figure 2 illustrates the characteristic **long-tail distribution** of movie popularity: a small cohort of blockbuster titles accumulates tens of thousands of ratings, whereas the majority of the catalog remains sparsely rated, making accurate recommendation of niche content particularly challenging.

4. Methodology & Data Pipeline

To handle the dataset's scale, we engineered a four-stage data pipeline:

1. **Parquet Serialisation:** Raw text files were converted to Apache Parquet format, reducing I/O overhead and enabling columnar, predicate-pushdown reads.

2. **Stratified Subsets:** A *Tiny* subset (500,000 ratings) was extracted to enable rapid local iteration without memory exhaustion.
3. **Temporal Train/Test Split:** Each user's ratings were sorted chronologically. The earliest 80% formed the training set; the most recent 20% served as the held-out test set -- preventing any temporal data leakage.
4. **Sparse Matrix Storage:** Interaction data was stored as SciPy `csr_matrix` objects, reducing RAM consumption by an order of magnitude compared to dense alternatives.

5. Model Design

Three recommendation algorithms were implemented and benchmarked:

- * **Item-Based Collaborative Filtering (ItemCF):** Recommends movies that are mathematically similar -- via cosine similarity of rating vectors -- to titles already rated positively by the target user.
- * **Alternating Least Squares (ALS):** An iterative matrix factorisation algorithm optimised for implicit and sparse data. ALS treats each observed rating as a *confidence* signal rather than an explicit preference value.
- * **Singular Value Decomposition (SVD):** Decomposes the user-movie matrix into *latent factor* vectors capturing hidden dimensions such as genre preference and directorial style. SVD is the core algorithm behind the original Netflix Prize winning submission.

6. Evaluation Metrics

- * **RMSE (Root Mean Square Error):** Quantifies the deviation of predicted star ratings from true ratings. Lower values indicate more accurate point predictions.
- * **MAP@10 (Mean Average Precision at 10):** Measures the fraction of the top-10 recommended titles that the user actually watched and rated ≥ 3.5 stars. Higher values reflect better ranking quality.

7. Experimental Results

The table below summarises benchmark performance across all three models trained on the *Tiny* subset. Bold entries highlight the best result in each metric column.

Model	RMSE (lower is better)	MAP@10 (higher is better)
SVD	0.9829	0.0016
ItemCF	1.2515	0.0000
ALS	3.7665	0.0040

Note: ALS produces abstract confidence scores rather than bounded 1-5 star predictions, causing its RMSE to appear inflated. However, it ranks relevant items effectively, as reflected by its superior MAP@10.

Key Finding: SVD achieves an RMSE below 1.0, making it the clear winner for accurate point-prediction of star ratings.

8. Recommendation Examples

To validate end-to-end system behaviour, we developed an interactive **Streamlit dashboard**. For User 0, the SVD model returned the following Top-3 recommendations:

1. *Arrested Development: Season 1*
2. *The Lord of the Rings: The Two Towers*
3. *Band of Brothers*

Explainability: The dashboard leverages SVD latent factor vectors to surface interpretable justifications. Given a query title (e.g., *The Matrix*), cosine similarity in the latent space identifies structurally similar sci-fi action films, providing users with transparent reasoning for each recommendation.

9. Key Insights & Limitations

- * **Memory Scalability:** Computing user-to-user similarity across 200,000+ users demands hundreds of gigabytes of RAM. Matrix factorisation (SVD) demonstrated far superior memory efficiency, making it the only viable approach at full dataset scale.
- * **Cold-Start Problem:** New users or newly added movies lack sufficient rating history for the model to infer meaningful preferences. Collaborative filtering algorithms are inherently blind to such sparse edge cases.

10. Future Improvements

The most impactful near-term enhancement would be adopting a **Hybrid Recommendation System** that blends collaborative filtering (SVD) with content-based filtering derived from movie metadata -- genres, plot descriptions, cast, and director. This hybrid strategy would:

- * Alleviate the cold-start problem for new users by falling back to content-based signals.
 - * Improve recommendation diversity and serendipity beyond the filter bubble.
 - * Increase coverage for long-tail niche titles that attract too few ratings for pure collaborative methods.
-